Context-Aware AI Scaffolding & Agent Architecture

Operational handbook for Claude Code & Google Antigravity: token economics, readiness gates, and work-order routing.

1. The Core Principle: Separate Decision-Making from Execution

Modern frontier models (Claude Opus, Gemini Pro) suffer from a critical cost bottleneck: **output token asymmetry**. High-tier models cost 20x to 40x more per output token than entry-tier models (Haiku, Flash-Lite). Generating raw implementation code with Opus burns session quotas rapidly. The solution is structural separation:

- **High-Tier Models (Opus / Gemini Pro):** Produce *few output tokens*. They plan, design, resolve architectural trade-offs, and generate structured Work Orders.
- **Low-Tier Models (Haiku / Gemini Flash):** Produce *many output tokens*. They write and refactor code strictly driven by deterministic Work Orders.
- **Readiness Routing:** Tasks are never dispatched based on perceived 'size' or 'complexity'. They are routed strictly by **how well-specified** they already are.

2. The 5-Tier Agent Architecture

Agent Name	Claude Tier	Antigravity Tier	Toolset	Mandate & Boundaries
bm-planner	Opus medium	Gemini Pro	Read-only (grep/glob/read)	Transforms vague/large tasks into verified Work Orders. **Never writes code.**
bm-coder	Haiku high	Gemini Flash	Full (edit/write/exec)	Writes code from a locked Work Order. **No design choices or ambiguity resolution.**
bm-mechanic	Haiku low	Gemini Flash-Lite	Full (atomic edits)	Zero-risk tasks: symbol renames, syntax formatting, progress/roadmap updates.
bm-light	Sonnet medium	Gemini Flash	Full (read/write/exec)	Focused fixes (1–2 files), technical judgment, code-verified documentation.
bm-deep	Opus xhigh	Gemini Pro	Full (isolated repro)	Strict gate: single isolated bug with repro or hard architectural decision. Rejects generic tasks.

3. The 5-Point Readiness Gate & Escalation Protocol

Before any task is routed to an execution model (mechanic, coder, light), the router evaluates a binary checklist:

1. **Target Files & Symbols:** Are specific file paths and function signatures explicitly identified?
2. **Diff Determinism:** Is the logic defined so strictly that two independent developers would produce identical diffs?
3. **Verifiable Command:** Is there an automated check (unit test, QEMU runner, compiler check, audit script)?
4. **Constrained Scope:** Does the change touch 3 files or fewer?
5. **Zero Pending Decisions:** Is the work completely free of unresolved design choices?

Rule: If any answer is **NO**, execution is blocked. The request routes to bm-planner. Agents terminate execution strictly with three possible statuses: **DONE** (verification passes), **BLOCKED** (ambiguous spec; returns to planner), or **FAILED x2** (test fails after 2 iterations; escalates to a higher tier).

4. Standard Work Order Template

```
### WORK ORDER: [WO-ID]
- Objective: [Single concise sentence defining the outcome]
- Target Files & Symbols: [Explicit paths and functions, e.g., src/bm/runtime.c:bm_init_cart()]
- Required Mutation: [Unambiguous diff specification leaving zero design latitude]
- Verification Command: [Deterministic execution, e.g., python tests/qemu_test.py -k ]
- Expected Result: [Exit code 0 and exact expected assertion output]
- Designated Agent: [bm-mechanic | bm-coder | bm-light]
```

5. Three-Tier Repository Scaffolding Structure

To prevent context rot, documentation must be divided across three distinct lifecycles:

- **Root Guardians (CLAUDE.md & GEMINI.md):** Kept under 60 lines. Contains only global invariants, build shortcuts, branching rules, and subagent routing. No release history.
- **Living Progress (docs/progress/.md):** Modular active sheets (80–120 lines). Tracks current state, open work, and unbreachable local rules.
- **Deep RFCs (docs/design/.md):** Speculative designs and long-term proposals kept outside the live progress index to prevent token overhead.

6. Continuous Guardrails & Quality Assurance

1. **Surgical Read Policy:** Never read full 300KB+ files. Use AST indexes (cindex.py) or targeted grep with offset/limit windows.
2. **Pre-Commit Verification:** Automated hooks trigger python scripts/docs_audit.py score to block commits if broken links or orphan docs are detected.
3. **Dual-Runtime Parity:** CLAUDE.md and GEMINI.md share the exact same state index, ensuring frictionless handoffs between Claude Code and Google Antigravity.